Test Case Document -- IBMS (Integrated Business Management Suite)

Field	Detail
Project Name	Integrated Business Management Suite (IBMS) v2.0
Team	Shivansh Srivastava (Product Developer), Prahallad Padhan (Product Owner), Ranveer Rai Khare (Scrum Master)
Sprint	Sprint 1
Date	April 12, 2026
Document Type	Test Case Specification
Test Framework	pytest (Python)

---

1. Test Environment Setup

1.1 Configuration (`pyproject.toml`)

Setting	Value
Python Path	`apps/ibms_core`
Test Paths	`apps/ibms_core/tests`
Python Target	3.10+

1.2 Test Fixtures (`conftest.py`)

The test suite uses a **mock Frappe ORM** approach to isolate unit tests from database dependencies:

Fixture/Mock	Description
`APP_PATH`	Dynamically adds `apps/ibms_core` to `sys.path` for imports
`fake_cache`	Mock Frappe cache with `get_value()` and `set_value()` stubs

Fixture/Mock	Description
`fake_frappe`	Mock Frappe namespace with `get_all()`, `cache()`, `session`, `has_permission()`, `throw()`
Module injection	Inserts mock `frappe` into `sys.modules` if not already loaded

Key Design Decision: Tests run without a database or Frappe bench installation by mocking the ORM layer completely.

---

2. Test Suite: JWT Authentication (`test_jwt_auth.py`)

Module Under Test: `ibms_core.security.jwt_auth`
Functions Tested: `issue_token()`, `validate_token()`, `decode_token()`

TC-AUTH-001: Token Issue and Validate Round Trip

Field	Detail
Test ID	TC-AUTH-001
Test Name	`test_issue_and_validate_token_round_trip`
Objective	Verify that a JWT token can be issued and validated with the same secret
Preconditions	jwt_auth module imported
Test Steps	1. Issue a token for `alice@example.com` with secret `secret-1` and TTL 3600s
	2. Validate the token using the same secret
Input Data	subject: `alice@example.com`, secret: `secret-1`, ttl: 3600
Expected Result	`validate_token()` returns `True`
Actual Result	Pass
Status	[x] PASS
Priority	Critical
Type	Unit -- Positive

TC-AUTH-002: Token Payload Contains Subject and Type

Field	Detail
Test ID	TC-AUTH-002
Test Name	`test_decode_token_contains_subject_and_type`
Objective	Verify that decoded JWT payload includes `sub` and `token_type` claims
Preconditions	jwt_auth module imported
Test Steps	1. Issue a token for `bob@example.com` with type `access`
	2. Decode the token with the same secret
	3. Assert `sub` field matches the input subject
	4. Assert `token_type` field matches `access`
Input Data	subject: `bob@example.com`, secret: `secret-2`, ttl: 3600, token_type: `access`
Expected Result	`payload["sub"] == "bob@example.com"` and `payload["token_type"] == "access"`
Actual Result	Pass
Status	[x] PASS
Priority	Critical
Type	Unit -- Positive

TC-AUTH-003: Token Validation Fails with Wrong Secret

Field	Detail
Test ID	TC-AUTH-003
Test Name	`test_validate_token_fails_with_wrong_secret`
Objective	Verify that token validation rejects tokens signed with a different secret (security check)
Preconditions	jwt_auth module imported
Test Steps	1. Issue a token for `eve@example.com` with secret `good-secret`
	2. Attempt to validate the token with `bad-secret`
Input Data	subject: `eve@example.com`, issue_secret: `good-secret`, validate_secret: `bad-secret`

Field	Detail
Expected Result	`validate_token()` returns `False`
Actual Result	Pass
Status	[x] PASS
Priority	Critical
Type	Unit -- Negative (Security)

---

3. Test Suite: AI Assistant Service (`test_ai_assistant.py`)

Module Under Test: `ibms_core.services.ai_assistant`
Functions Tested: `ask_assistant()`, `recommend_actions()`

Mock Data Setup

The tests use a `FakeFrappe` class to simulate database responses:

DocType	Mock Data
KPI Snapshot	`risk_exposure: 32.1`, `revenue_run_rate: ?12,500,000` (2026-03-13)
AI Recommendation	`REC-0001`, code: `WF-REDUCE-APPROVAL-HOPS`, confidence: `0.78`

TC-AI-001: AI Assistant Risk Query

Field	Detail
Test ID	TC-AI-001
Test Name	`test_assistant_risk_query`
Objective	Verify that the AI assistant returns risk metrics when queried about risk details
Preconditions	`ai_assistant.frappe` patched with `FakeFrappe` mock
Test Steps	1. Monkeypatch `ai_assistant.frappe` with `FakeFrappe()`
	2. Call `ask_assistant("show risk details", "Default Company", "alice@example.com")`

Field	Detail
	3. Assert `risk_metrics` key exists in the result
	4. Assert at least 1 risk metric record returned
Input Data	query: `"show risk details"`, company: `"Default Company"`, user: `"alice@example.com"`
Expected Result	Result contains `risk_metrics` with ? 1 entry
Actual Result	Pass
Status	[x] PASS
Priority	High
Type	Unit -- Positive

TC-AI-002: AI Recommendations Payload

Field	Detail
Test ID	TC-AI-002
Test Name	`test_recommendations_payload`
Objective	Verify that `recommend_actions()` returns correct recommendation data
Preconditions	`ai_assistant.frappe` patched with `FakeFrappe` mock
Test Steps	1. Monkeypatch `ai_assistant.frappe` with `FakeFrappe()`
	2. Call `recommend_actions("Default Company")`
	3. Assert `count == 1`
	4. Assert first recommendation code is `WF-REDUCE-APPROVAL-HOPS`
Input Data	company: `"Default Company"`
Expected Result	`count: 1`, recommendation code: `WF-REDUCE-APPROVAL-HOPS`
Actual Result	Pass
Status	[x] PASS
Priority	High
Type	Unit -- Positive

4. API Integration Test Cases (Manual / curl)

These test cases verify the running server endpoints and were validated manually during this sprint.

TC-API-001: Health Check Endpoint

Field	Detail
Test ID	TC-API-001
Test Method	`curl http://localhost:8000/api/health`
Objective	Verify the health endpoint returns a healthy status
Expected Result	`{"status": "healthy", "version": "2.0.0", ...}`
Actual Result	`status: healthy`, `redis: false` (fallback mode), `uptime_seconds: N`, `error_rate: 0.0`
Status	[x] PASS

TC-API-002: Dashboard KPI Endpoint

Field	Detail
Test ID	TC-API-002
Test Method	`curl http://localhost:8000/api/dashboard`
Objective	Verify dashboard returns real-time KPI data
Expected Result	JSON with `revenue`, `net_margin`, `risk_exposure`, `forecast_accuracy` fields
Status	[x] PASS

TC-API-003: User Registration

Field	Detail
Test ID	TC-API-003
Test Method	`curl -X POST http://localhost:8000/api/auth/register -H "Content-Type: application/json" -d '{"username":"testuser","password":"Test@12345","email":"test@example.com"}'`

Field	Detail
Objective	Verify new users can register
Expected Result	`201 Created` with user object and access token
Status	[x] PASS

TC-API-004: User Login

Field	Detail
Test ID	TC-API-004
Test Method	`curl -X POST http://localhost:8000/api/auth/login -H "Content-Type: application/json" -d '{"username":"testuser","password":"Test@12345"}'`
Objective	Verify registered users can log in and receive JWT tokens
Expected Result	`200 OK` with `access_token`, `csrf_token`, and `Set-Cookie: refresh_token`
Status	[x] PASS

TC-API-005: Risk Scoring

Field	Detail
Test ID	TC-API-005
Test Method	`curl -X POST http://localhost:8000/api/risk/composite -H "Content-Type: application/json" -d '{"factors":{"amount":80,"behavior":50,"compliance":30}}'`
Objective	Verify composite risk score calculation
Expected Result	Weighted score: `(0.5 × 80) + (0.3 × 50) + (0.2 × 30) = 61.0`
Status	[x] PASS

TC-API-006: Fraud Detection

Field	Detail

Field	Detail
Test ID	TC-API-006
Test Method	`curl -X POST http://localhost:8000/api/fraud/detect -H "Content-Type: application/json" -d '{"transaction":{"amount":100000,"vendor":"unknown"}}'`
Objective	Verify Isolation Forest fraud scoring returns a score with review flag
Expected Result	JSON with `fraud_score` (0-1) and `flag_for_review` boolean
Status	[x] PASS

TC-API-007: Compliance Check

Field	Detail
Test ID	TC-API-007
Test Method	`curl -X POST http://localhost:8000/api/compliance/check -H "Content-Type: application/json" -d '{"transaction":{"amount":600000}}'`
Objective	Verify high-value transactions without approval are flagged
Expected Result	`passed: false`, violations: `["MISSING_HIGH_VALUE_APPROVAL"]`
Status	[x] PASS

TC-API-008: Rate Limiting

Field	Detail
Test ID	TC-API-008
Test Method	Rapid sequential requests to any endpoint
Objective	Verify rate limiting returns 429 when threshold exceeded
Expected Result	`429 Too Many Requests` after exceeding the limit
Status	[x] PASS

5. Test Coverage Summary

Module	Unit Tests	Integration Tests	Total	Coverage
JWT Authentication	3	1	4	Core flows covered
AI Assistant Service	2	--	2	Query + recommendations
API Health/System	--	2	2	Health + dashboard
Auth (Login/Register)	--	2	2	Registration + login
Risk & Fraud	--	3	3	Risk, fraud, compliance
Rate Limiting	--	1	1	Throttle verification
Total	5	9	14	--

Running the Tests

```
# Run all unit tests
cd Integrated-Management-Business-Suite
pytest apps/ibms_core/tests/ -v

# Run specific test file
pytest apps/ibms_core/tests/test_jwt_auth.py -v

# Run with coverage report
pytest apps/ibms_core/tests/ --cov=ibms_core --cov-report=term-missing
```

6. Known Limitations & Future Test Improvements

Area	Current Status	Planned Improvement
WebSocket tests	Not automated	Add async WebSocket client tests
2FA flow (setup -> confirm)	Manual only	Automate TOTP verification tests
Token refresh rotation	Manual only	Add cookie-based refresh test
Load/performance tests	Not started	Add Locust/k6 load testing scripts
Frontend (app.js)	Not tested	Add Playwright/Cypress E2E tests

Area	Current Status	Planned Improvement
Database integration	Mocked	Add Frappe test bench integration suite

Document Version 1.0 -- Sprint 1 Review